



程序设计原理

文件输入输出

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤



Objectives

- To use ofstream for output and ifstream for input .
- To test whether a file exists .
- To test the end of a file .
- To write data in a desired format .
- To read and write data using the getline, get, and put functions .
- To use an fstream object to read and write data .
- To open a file with specified modes .
- To use the eof(), fail(), bad(), and good() functions to test stream states .



Writing Data to a File

The ofstream class can be used to write primitive data type values, arrays, strings, and objects to a text file. `TextFileOutput.cpp` demonstrates how to write data. The program creates an instance of ofstream and writes two lines to the file “scores.txt”. Each line consists of first name (a string), middle name initial (a character), last name (a string), and score (an integer).



Writing Data to a File

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  int main()
6  {
7      ofstream output;
8
9      // Create a file
10     output.open("scores.txt");
11
12     // Write two lines
13     output << "John" << " " << "T" << " " << "Smith"
14         << " " << 90 << endl;
15     output << "Eric" << " " << "K" << " " << "Jones"
16         << " " << 85;
17
18     output.close();
19
20     cout << "Done" << endl;
21
22     return 0;
23 }
```



Writing Data to a File

```
output << "John" << " " << "T" << "Smith" << " " << 90 << endl;
```

scores.txt
file

John T Smith 90
Eric K Jones 85

```
output << "Eric" << " " << "K" << "Jones" << " " << 85 << endl;
```



Close File

The close() function (line 18) must be used to close the stream for the object. If this function is not invoked, the data may not be saved properly in the file.



File Exists?

If a file already exists, the contents of the file will be destroyed without warning.



Absolute Filename

Every file is placed in a directory in the file system. An *absolute file* name contains a file name with its complete path and drive letter. For example, `c:\example\scores.txt` is the absolute file name for the file `scroes.txt` on the Windows operating system. Here `c:\example` is referred to as the *directory path* for the file. Absolute file names are machine-dependent. On Unix, the absolute file name may be `/home/liang/example/scores.txt`, where `/home/liang/example` is the directory path for the file `scores.txt`.



\ in file names

The directory separator for Windows is a backslash (\). The backslash is a special character in C++ and should be written as `\\` in a string literal (see Table 2.4). For example,

```
output.open("c:\\example\\scores.txt");
```



Relative Filename

Absolute file name is platform dependent. It is better to use relative file name without drive letters. The directory of the relative filename can be specified in the IDE (Integrated Development Environment) if you use an IDE to run C++. For example, the default directory for data files is `\windows\Debug_Build` for Borland C++Builder.



Reading Data from a File

The ifstream class can be used to read data from a text file. `TextFilein.cpp` demonstrates how to read data. The program creates an instance of ifstream and reads data from the file `scores.txt`. `scores.txt` was created in the preceding example.



Reading Data from a File

```
1 #include <iostream>
2 #include <fstream>
3 using namespace std;
4
5 int main()
6 {
7     ifstream input;
8
9     // Open a file
10    input.open("scores.txt");
11
12    // Read data
13    char firstName[80];
14    char mi;
15    char lastName[80];
16    int score;
17    input >> firstName >> mi >> lastName >> score;
18    cout << firstName << " " << mi << " " << lastName << " "
19         << score << endl;
20
21    input >> firstName >> mi >> lastName >> score;
22    cout << firstName << " " << mi << " " << lastName << " "
23         << score << endl;
24
25    input.close();
26
27    cout << "Done" << endl;
28
29    return 0;
30 }
```



Testing File Existence

If the file does not exist, your program will run and produce incorrect results. Can your program check whether a file exists? Yes. You can invoke the fail() function immediately after invoking the open function. If fail() returns true, it would indicate that the file does not exist.



Testing End of File

TextFilein.cpp reads two lines from the data file. If you don't know how many lines are in the file and want to read them all, how do you know the end of file? You can invoke the eof() function on the input object to detect it. TestEndOfFile.cpp revises TextFilein.cpp to read all lines from the file scores.txt.



Testing End of File

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  int main()
6  {
7      ifstream input;
8      // Open a file
9      input.open("scores.txt");
10
11     if (input.fail()){
12         cout << "File does not exist" << endl;
13         cout << "Exit program" << endl;
14         return 0;
15     }
16     // Read data
17     char firstName[80];
18     char mi;
19     char lastName[80];
20     int score;
21     while (!input.eof()){ // Continue if not end of file
22         input >> firstName >> mi >> lastName >> score;
23         cout << firstName << " " << mi << " " << lastName
24             << " " << score << endl;
25     }
26     input.close();
27     cout << "Done" << endl;
28     return 0;
29 }
```



Know Data Format

To read data correctly, you need to know exactly how data is stored. For example, the program in TestEndOfFile.cpp would not work if the score is a double value with a decimal point.



Formatting Output

You have used the stream manipulators to format output to the console in §3.11, “Formatting Output.” You can use the same stream manipulator to format output to a file. WriteFormatData.cpp gives an example that formats the student records to the file named formattedscores.txt.



Formatting Output

```
1  #include <iostream>
2  #include <iomanip>
3  #include <fstream>
4  using namespace std;
5
6  int main()
7  {
8      ofstream output;
9
10     // Create a file
11     output.open("formattedscores.txt");
12
13     // Write two lines
14     // Write two lines
15     output << setw(6) << "John" << setw(2) << "T" << setw(6) << "Smith"
16     << " " << setw(4) << 90 << endl;
17     output << setw(6) << "Eric" << setw(2) << "K" << setw(6) << "Jones"
18     << " " << setw(4) << 85;
19
20     output.close();
21
22     cout << "Done" << endl;
23
24     return 0;
25 }
```



getline, get and put

There is a problem to read data using the stream extraction operator ($\geq\geq$). Data are delimited by whitespace(space, TAB, Enter). What happens if the whitespace characters are part of a string? In §7.9.3, “Reading Strings,” you learned how to use the getline function to read a string with whitespace. You can use the same function to read strings from a file. Recall that the syntax for the getline function is

getline(char array[], int size, char delimiterChar)



getline, get and put

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  int main()
6  {
7      ifstream input;
8
9      // Open a file
10     input.open("state.txt");
11
12     if (input.fail()){
13         cout << "File does not exist" << endl;
14         cout << "Exit program" << endl;
15         return 0;
16     }
17
18     // Read data
19     char city[40];
20     while (!input.eof()){ // Continue if not end of file
21         input.getline(city, 40, '#');
22         cout << city << endl;
23     }
24
25     input.close();
26
27     cout << "Done" << endl;
28     return 0;
29 }
```



getline, get and put

Two other useful functions are get and put. You can invoke the `get` function on an input object to read a character and invoke the `put` function on an output object to write a character.



getline, get and put

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  int main(){
6      const int FILENAME_SIZE = 40;
7      // Enter a source file
8      cout << "Enter a source file name: ";
9      char inputFilename[FILENAME_SIZE];
10     cin >> inputFilename;
11     // Enter a target file
12     cout << "Enter a target file name: ";
13     char outputFilename[FILENAME_SIZE];
14     cin >> outputFilename;
15     ifstream input;
16     ofstream output;
17     // Open a file
18     input.open(inputFilename);
19     output.open(outputFilename);
20     if (input.fail()){
21         cout << inputFilename << " does not exist" << endl;
22         cout << "Exit program" << endl;
23         return 0;
24     }
25     while (!input.eof()) // Continue if not end of file
26         output.put(input.get());
27     input.close();
28     output.close();
29     cout << "\nCopy Done" << endl;
30     return 0;
31 }
```



getline()

getline **does not skip leading whitespace characters** such as blanks and newlines

getline reads successive characters (including blanks) into the string, and stops when it reaches the newline character ‘\n’

the newline is consumed by getline, but is not stored into the string variable



get()

get **does not skip leading whitespace** characters such as blanks and newlines

get reads successive characters (including blanks) into the array, and **stops when it either has read count characters, or it reaches the newline character '\n', whichever comes first**

get **appends the null character** to str

if it is reached, **newline is not consumed** by get, but remains waiting in the input stream



getline, get and put

```
cin >> firstname >> score;  
cin.get();  
cin.getline(lastname, 80);  
cin.getline(mi, 80);  
cin.get(comments, 80);  
cin.getline(comments2, 80);
```

```
cout << "firstname:" << firstname << endl;  
cout << "score:" << score << endl;  
cout << "lastname:" << lastname << endl;  
cout << "mi:" << mi << endl;  
cout << "comments:" << comments << endl;  
cout << "comments2:" << comments2 << endl;
```

```
tian 98  
tianjin daxue  
nankai daxue  
zhongguo tianjin
```

```
firstname:tian  
score:98  
lastname:tianjin daxue  
mi:nankai daxue  
comments:zhongguo tianjin  
comments2:
```



`fstream` and File Open Modes

In the preceding sections, you used the `ofstream` to write data and the `ifstream` to read data. Alternatively, you can also use the `fstream` class to create an input stream or output stream. It is convenient to use `fstream` if your program needs to use the same stream object for both input and output. To open an `fstream` file, you have to specify a file mode to tell C++ how the file will be used. The file modes are listed in Table 12.1.



File Open Modes

Mode	Description
<code>ios::in</code>	Opens a file for input.
<code>ios::out</code>	Opens a file for output.
<code>ios::app</code>	Appends all output to the end of the file.
<code>ios::ate</code>	Opens a file for output. If the file already exists, move to the end of the file. Data can be written anywhere in the file.
<code>ios::trunc</code>	Discards the file's contents if the file already exists. (This is the default action for <code>ios::out</code>).
<code>ios::binary</code>	Opens a file for binary input and output.



Combining Modes

Several modes can be combined together using the `|` operator. This is a bitwise inclusive OR operator. See function in a derived class is called *overriding a function*. Appendix D, “Bit Operations,” for more details. For example, to open an output file named `city.txt` for appending data, you can use the following statement:

```
stream.open("city.txt", ios::out | ios::app);
```



Combining Modes

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  int main()
6  {
7      fstream inout;
8      // Create a file
9      inout.open("city.txt", ios::out);
10     // Write cities
11     inout << "Dallas" << " " << "Houston" << " " << "Atlanta" << " ";
12     inout.close();
13
14     // Append to the file
15     inout.open("city.txt", ios::out | ios::app);
16     // Write cities
17     inout << "Savannah" << " " << "Austin" << " " << "Chicago";
18     inout.close();
19
20     char city[20];
21     // Open the file
22     inout.open("city.txt", ios::in);
23     while (!inout.eof()){ // Continue if not end of file
24         inout >> city;
25         cout << city << " ";
26     }
27     inout.close();
28     return 0;
29 }
```



Exercise

[题目描述](#)[我的提交](#)[题目讨论](#)

字符串修改

[好评](#) [差评](#)[文件 IO](#)[暂无标签](#)[编辑题目](#) [题目统计](#) [全部提交](#)

输入文件 string.in

1000MS, 其他语言 2000MS

输出文件 string.out

256MB, 其他语言 512MB

出题人: root

描述

给两个字符串 s, t , 将 s 串的 k 位置以后替换为字符串 t , 并将新串翻转。

输入描述

第一行输入一个整数 $k(0 \leq k < |s|)$ 。

接下来两行, 每行一个字符串, 代表 s 和 t 。

输出描述

输出替换后的字符串。

用例输入 1

```
3
abc def
def ghi
```

用例输出 1

```
ihg fedcba
```

提示

 $1 \leq |s|, |t| \leq 100$



Exercise

语言:

C++



```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     freopen("string.in","r",stdin);
7     freopen("string.out","w",stdout);
8     int k;
9     char s[110],t[110];
10    cin>>k;
11
12    getchar();
13
14    cin.get(s,100);
15    getchar();
16
17    cin.getline(t,100);
18
19    string l="";
20    for(int i=0;i<k;i++)
21        l=l+s[i];
22    l+=t;
23    for(int i=l.size()-1;i>=0;i--)
24        cout<<l[i];
25    cout<<endl;
26    return 0;
27 }
```